Liga de Mercado Financeiro
Criada por Estudantes, reconhecida pelo Mercado.

BACKTESTING
VIESES & BACKTRADER

Núcleo de Análise Quantitativa

Liga de Mercado Financeiro · UFU · 2026.1

DIRETOR DE PESQUISA
Arthur Pires Lopes — Eng. Elétrica

GERENTE QUANT
João Emanuel Zilli — Eng. da Computação

ASSESSOR
Gustavo Kono — Eng. da Computação

ASSESSOR
Wilson Ribeiro — Eng. da Computação

ASSESSOR
Thiago Leal — Ciências Econômicas



Diretoria de Pesquisa · 2026.1

? Seu backtest deu Sharpe de 3.2 e retorno de 80% ao ano. O que pode estar errado?

Um resultado bom demais é quase sempre um sinal de que algo está errado no processo. Os três vieses desta aula são os mais comuns, e os mais silenciosos.

Look-Ahead Bias

Usou informação do futuro no passado

Survivorship Bias

Testou só nos ativos que sobreviveram

Overfitting

Decorou o passado em vez de aprender

Look-Ahead Bias: futuro para prever o futuro

Look-ahead bias ocorre quando o modelo usa, no momento da decisão, uma informação que só estaria disponível depois. O backtest parece funcionar porque 'trapaceou', mas em produção, essa informação não existe ainda.

O que acontece na prática

Dia D — Decisão: comprar ou vender?

↓

Dia D+1 — Dado publicado (resultado, dividendo...)

↓

Bug — Modelo usou D+1 na decisão de D

❌ **Com look-ahead bias**

```
# sinal gerado com dado do DIA SEGUINTE
sinal = (media.shift(-1) > 0).astype(int)
```

✅ **Sem look-ahead bias**

```
# sinal gerado com dado de ONTEM (já disponível)
sinal = (media.shift(1) > 0).astype(int)
```

Casos comuns de look-ahead bias:

→ Usar preço de fechamento para gerar sinal do mesmo dia
→ Normalizar com média/std do período completo antes de treinar
→ Usar .shift(-1) por engano no sinal de entrada

⚠️ Se você usou .shift(-1) em vez de .shift(1) no seu sinal, está olhando para amanhã. Resultado: backtest perfeito, produção zero.

?

Você testou sua estratégia em todas as ações do Ibovespa. Mas o Ibovespa de hoje é o mesmo de 2010?

O Ibovespa de 2010 tinha empresas que faliram, foram deslistadas ou saíram do índice. Se você usou a composição atual, testou só nos vencedores, o survivorship bias.

Ibovespa 2010

PETR4	VALE3	OGX
PDVL4	ITUB4	OGXP3
WEGE3	BRT3	BBDC4

Você vê isso

→

Ibovespa hoje (se você usar a composição atual)

PETR4	VALE3	ITUB4
WEGE3	BBDC4	

Mas testa só isso

Survivorship Bias: testar só nos vencedores

Se sua base de dados contém só ativos que ainda existem hoje, você removeu silenciosamente todas as empresas que faliram ou foram deslistadas. Seu backtest nunca vai comprar uma OGX, mas sua estratégia ao vivo compraria.

Impacto no resultado

Retorno médio (com survivorship)
+22% a.a.

Retorno médio (universo completo)
+9% a.a.

Diferença estimada do viés
+13 p.p.

Como acontece

01

Você baixa os tickers do Ibovespa atual e testa de 2010 até hoje. As empresas que saíram do índice (ou faliram) simplesmente não existem na sua base.

Como detectar

02

Verifique se sua base tem ativos que foram deslistados. Se não tiver nenhuma empresa falida no universo, você tem survivorship bias.

Como corrigir

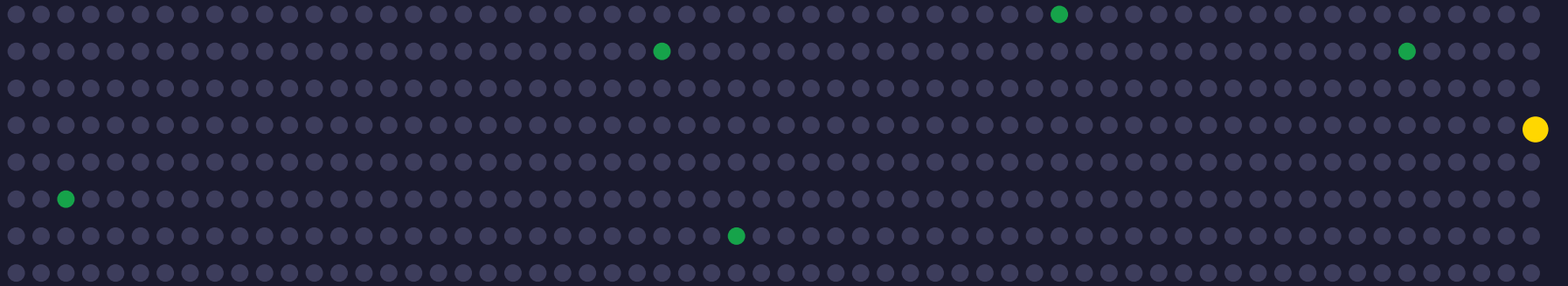
03

Use bases de dados com histórico point-in-time, que preservam a composição do índice em cada data. Ex: Refinitiv, Compustat, B3 histórico.

?

Você testou 500 combinações de parâmetros e a melhor deu Sharpe 4.1. Esse resultado é confiável?

Não. Ao testar 500 combinações, você tem 500 tentativas de encontrar uma que funcione por puro acaso no histórico. Isso chama overfitting — ou p-hacking em pesquisa quantitativa.



Cada ponto = 1 combinação de parâmetros testada

🏅 Melhor combinação (Sharpe 4.1)

Overfitting: decorar o passado em vez de aprender

Overfitting é quando o modelo se ajusta tão bem ao histórico que perde poder preditivo. Ele aprendeu o ruído específico daquele período — não um padrão real. Em produção, o resultado desmorona.

Sinal clássico de overfitting

In-sample (treino)

Sharpe: 4.1 | Retorno: +82%

Out-of-sample (teste)

Sharpe: 0.3 | Retorno: +4%

A colapso de Sharpe 4.1 → 0.3 entre treino e teste é o sintoma clássico. O modelo decorou o passado — não aprendeu um padrão generalizável.

01 Walk-forward analysis

Divide o histórico em janelas: treina em uma, testa na próxima, avança. Replica como a estratégia rodaria ao vivo.

02 Out-of-sample obrigatório

Reserve 20-30% do histórico e só olhe para ele depois de finalizar todos os parâmetros. Nunca use no desenvolvimento.

03 Menos parâmetros livres

Cada parâmetro livre é uma oportunidade de overfitting. Estratégias robustas funcionam com configurações simples e amplas.

Os três inimigos de qualquer backtest

Look-Ahead Bias 01

CAUSA

Usar informação futura na decisão presente

SINTOMA

Backtest perfeito, produção zero

CORREÇÃO

Usar .shift(1) no sinal. Nunca usar dado do mesmo candle para decidir.

Survivorship Bias 02

CAUSA

Testar só em ativos que ainda existem hoje

SINTOMA

Retorno inflado em 10–15 p.p. ao ano

CORREÇÃO

Usar base point-in-time com ativos deslistados e falidos incluídos.

Overfitting 03

CAUSA

Otimizar parâmetros excessivamente no histórico

SINTOMA

Sharpe alto in-sample, colapso out-of-sample

CORREÇÃO

Walk-forward, out-of-sample e poucos parâmetros livres.

?

Você entendeu os vieses. Como rodar um backtest real em Python sem cometê-los?

Backtrader é um framework Python que estrutura o backtest de forma que os vieses mais comuns se tornam difíceis de cometer — o dado de cada candle só é acessível após o fechamento.

Event-driven

Simula tick a tick — cada dado novo dispara a lógica da estratégia

Sem look-ahead

Só acessa o candle fechado. bar[0] = hoje (fechado), bar[-1] = ontem

Indicadores built-in

SMA, EMA, RSI, Bollinger Bands já implementados e anti-bias

Métricas automáticas

Sharpe, drawdown, retorno anual calculados automaticamente

Estrutura de uma estratégia:

Toda estratégia no Backtrader herda de bt.Strategy e implementa três métodos. Entender esses três métodos é tudo que você precisa para rodar o primeiro backtest.

```python
class MinhaEstrategia(bt.Strategy):

    def __init__(self):
        # 1. Definir indicadores
        self.sma20  = bt.ind.SMA(period=20)
        self.sma200 = bt.ind.SMA(period=200)

    def next(self):
        # 2. Lógica de trading (roda a cada candle)
        if self.sma20[0] > self.sma200[0]:
            self.buy()
        else:
            self.sell()

    def notify_trade(self, trade):
        # 3. Callback de trade executado (opcional)
        if trade.isclosed:
            print(f"P&L: {trade.pnl:.2f}")
```

__init__: indicadores 01

Chamado uma vez antes do backtest. Declare aqui todos os indicadores. O Backtrader garante que só usam dados passados — zero look-ahead.

next(): lógica de trading 02

Chamado a cada novo candle fechado. self.sma20[0] = valor de hoje, self.sma20[-1] = ontem. Impossível acessar o futuro.

bar[0] vs bar[-1] 03

Indexação intuitiva e anti-bias: [0] é o candle atual (já fechado), [-1] é o anterior. Nunca há acesso ao [+1] — ele ainda não existe.

Rodando o Cerebro: dados, estratégia e métricas

O Cerebro é o orquestrador do Backtrader. Você adiciona dados, a estratégia e os analisadores de métricas — e ele roda a simulação completa.

```python
# 1. Adicionar dados
data = bt.feeds.PandasData(dataname=df)
cerebro.adddata(data)

# 2. Adicionar a estratégia
cerebro.addstrategy(MinhaEstrategia)

# 3. Capital inicial e comissão
cerebro.broker.setcash(100_000)
cerebro.broker.setcommission(commission=0.001)

# 4. Analisadores de métricas
cerebro.addanalyzer(bt.analyzers.SharpeRatio,
    _name="sharpe", riskfreerate=0.145)
cerebro.addanalyzer(bt.analyzers.DrawDown,_name="dd")

# 5. Rodar!
results = cerebro.run()
cerebro.plot()
```

bt.feeds.PandasData

Converte qualquer DataFrame com OHLCV para o formato do Backtrader. Colunas precisam se chamar open, high, low, close, volume. 01

cerebro.broker

Configura capital inicial e comissão por trade. Sem isso, você ignora custos reais — outro viés comum em backtests. 02

addanalyzer

Sharpe, Drawdown, Returns calculados automaticamente ao final. Acesse com results[0].analyzers.sharpe.get_analysis(). 03